

# Serverless Microservices Architecture

## Introduzione

### Definizione

I microservizi serverless sono un tipo di architettura a microservizi in cui **L'INFRASTRUTTURA È GESTITA DA UN FORNITORE DI CLOUD** e i microservizi sono eseguiti in un ambiente di elaborazione serverless.

★ *"Ciò significa che i microservizi vengono **SCALATI AUTOMATICAMENTE** per soddisfare la domanda dell'applicazione, e lo sviluppatore **NON** deve preoccuparsi del provisioning e della gestione dei server."*

### Le piattaforme

| Provider        | Servizio FaaS          |
|-----------------|------------------------|
| AWS             | AWS Lambda             |
| Microsoft Azure | Azure Functions        |
| Google Cloud    | Google Cloud Functions |

*"Consentono agli sviluppatori di scrivere e distribuire microservizi che vengono eseguiti **IN RISPOSTA A EVENTI SPECIFICI** o API, come una richiesta HTTP o una modifica a un database."*

**Definizione: FaaS (Function as a Service)** — il modello in cui l'unità di deployment non è un'applicazione né un container, ma **una singola funzione**. La piattaforma la esegue **solo quando arriva un evento**, e **paghi solo il tempo di esecuzione effettivo**.

### ★ I 6 passi per implementare un microservizio serverless

1. SCRIVI il codice in un linguaggio supportato (Node.js, Python, Java, C#...)
2. IMPACCHETTA codice e dipendenze in un pacchetto distribuibile (es. file .zip)
3. CREA una funzione AWS Lambda e CARICA il pacchetto
4. CONFIGURA il TRIGGER: quale evento attiva la funzione  
(es. una richiesta HTTP dall'API Gateway)
5. TESTA ed esegui il DEBUG (console AWS Lambda o CLI)
6. DISTRIBUISCI in produzione e MONITORA con AWS CloudWatch

Nota il passo 4: **la funzione non "gira"** come un servizio in ascolto. **Esiste, e viene invocata da un evento**. È la differenza concettuale fondamentale rispetto a un microservizio containerizzato.

---

# AWS Serverless Microservices with Patterns & Best Practices

I servizi AWS che verranno usati, raggruppati per categoria.

## ★ Le 6 categorie di servizi

### 1. Compute (calcolo) → AWS Lambda

*"Un servizio serverless che **non si preoccupa di server e macchine**. Ci si preoccupa **SOLO di caricare il codice**. AWS si occupa di **scalare la macchina, del provisioning e di distribuire il codice quando il traffico arriva**."*

★ *"Quando si usa AWS Lambda, **TUTTE le attività relative al server vengono NASCOSTE**."*

### 2. Database → Amazon DynamoDB

*"Un **database NoSQL key-value** che **SCALA ORIZZONTALMENTE**, ed è il database più diffuso nei servizi AWS Serverless."*

Richiama la Lezione 10: DynamoDB è un classico sistema **AP** con sharding automatico.

### 3. Messaging & Application Integration → Amazon EventBridge + Amazon SQS

*"Per costruire **un'architettura guidata dagli eventi**. Implementeremo il **pattern di integrazione FANOUT** e il **pattern publish/subscribe**."*

*"Questo è anche **un modo per DISACCOPPIARE i microservizi**."*

Richiama direttamente la Lezione 9: il fanout è il modello 1-a-molti del pub/sub.

### 4. API Management → Amazon API Gateway

*"Serve **fondamentalmente per esporre API RESTful senza pensare ad alcun server e scalare i metodi API**."*

Richiama la Lezione 8: stesso pattern API Gateway, ma **completamente gestito**.

### 5. Infrastructure as Code → AWS CloudFormation + AWS CDK

*"Svilupperemo la nostra infrastruttura con **AWS CDK**, e il **CDK usa CloudFormation sotto il cofano, a partire da codice TypeScript**. Ecco perché usiamo entrambi."*

**Definizione:** AWS CDK (Cloud Development Kit) — permette di definire l'infrastruttura **in un linguaggio di programmazione reale** (TypeScript, Python, C#, Java) invece che in YAML/JSON. Il CDK poi **genera** i template CloudFormation. È lo stesso concetto di **Pulumi** visto nella Lezione 14.

### 6. Monitoring → Amazon CloudWatch

*"Esamineremo **i log dei microservizi Lambda**. È molto importante **seguire i tipi di invocazione [e avere] funzioni di REGISTRAZIONE CENTRALIZZATA**."*

Richiama la Lezione 15: **logging centralizzato**, ma come servizio gestito invece di Elastic Stack self-hosted.

---

## ★ La mappatura ai tre tipi di comunicazione

Questa è la parte più interessante della sezione, perché **riprende le Lezioni 7 e 9** e le traduce in servizi AWS.

| # | Tipo di comunicazione          | Servizi AWS                     | Microservizi coinvolti             |
|---|--------------------------------|---------------------------------|------------------------------------|
| 1 | Sincrona (RESTful)             | API Gateway + Lambda + DynamoDB | Product e Basket                   |
| 2 | Asincrona (event-driven)       | Amazon EventBridge              | Basket pubblica → Ordering consuma |
| 3 | Event Source Mapping (polling) | Amazon SQS                      | Ordering fa polling dalla coda     |

---

### Tipo 1 — Comunicazione sincrona / RESTful

*"Svilupperemo **microservizi RESTful** con **API Gateway** + **funzioni Lambda** + **DynamoDB**. Esposizione di **API REST** ed esecuzione di operazioni **CRUD** con **AWS Lambda** e **API Gateway**, e **persistenza dei dati** con **DynamoDB**."*

Client → [API Gateway] → [Lambda: Product] → [DynamoDB]  
→ [Lambda: Basket] → [DynamoDB]

### Tipo 2 — Comunicazione asincrona

*"Useremo **Amazon EventBridge** per la comunicazione asincrona event-driven. I **microservizi Basket PUBBLICANO** un evento in modo asincrono, e questo viene consumato dai **microservizi Ordering** usando la coda **SQS**."*

### Tipo 3 — Event Source Mapping (polling)

*"In un evento di polling, i **consumer** inviano [richieste] al **producer** in **BATCH**."*

*"Useremo **AWS SQS** per **DISACCOMPIARE** i **microservizi** ed elaborare gli eventi in modo asincrono usando le **CODE**. Svilupperemo **SQS** con **Ordering Lambda**, che consuma **fondamentalmente** eseguendo il **POLLING** degli eventi dalla coda **Ordering**."*

**Definizione: Event Source Mapping** — un meccanismo AWS in cui **Lambda** stessa fa il **polling** di una sorgente (SQS, Kinesis, DynamoDB Streams) e **invoca la funzione** con un **batch di messaggi**. Lo sviluppatore non scrive il loop di polling: lo fa la piattaforma.

## ★ Il flusso completo del checkout

```
1. Client → [API Gateway] → [Lambda Basket] → checkout
                                     ↓ publish
2.                                [Amazon EventBridge] ← event bus serverless
                                     ↓ rule
3.                                [Amazon SQS Queue]    ← coda Ordering
                                     ↓ polling (event source mapping)
4.                                [Lambda Ordering] → [DynamoDB]
```

"Ovviamente, i microservizi Ordering **espongono API in modo sincrono**, e l'intera architettura sarà sviluppata con **INFRASTRUCTURE AS CODE**, usando **AWS CDK** e **CloudFormation**."

---

## AWS Lambda as a Microservice

### Il ripasso: cosa sono i microservizi

*"I microservizi sono **piccoli servizi aziendali che possono lavorare insieme e possono essere distribuiti in modo AUTONOMO e INDIPENDENTE**. Comunicano **attraverso la rete**."*

*"Hanno **un proprio stack tecnologico, incluso il modello di gestione dei database**, comunicano tramite **una combinazione di API REST, streaming avanzato o message broker**, e sono **organizzati in base alle CAPACITÀ AZIENDALI**, con la linea di separazione dei servizi **spesso indicata come BOUNDED CONTEXT**."*

Nota il richiamo diretto alla **Lezione 6** (decomposizione per business capability e bounded context) — il principio **non cambia** nel serverless.

### Dove entra Lambda

**Definizione: AWS Lambda** — *"un servizio che consente di eseguire funzioni nel cloud **COMPLETAMENTE SENZA SERVER, ELIMINANDO LA COMPLESSITÀ OPERATIVA**."*

*"Si carica il codice su Lambda, che si occupa di tutto ciò che serve per **ESEGUIRE e SCALARE** l'esecuzione, e soddisfare i requisiti di **ALTA DISPONIBILITÀ**."*

*"Supporta **diversi linguaggi di programmazione**. Si integra con l'API Gateway, permette di invocare funzioni con le chiamate API, e rende la vostra architettura **completamente serverless**."*

## ★ Il vantaggio architetturale centrale

*"L'intero servizio è costruito a partire dai **SERVIZI GESTITI**, il che:"*

- ★ *"**ELIMINA L'ONERE ARCHITETTONICO** di progettare per la **SCALABILITÀ o l'ALTA DISPONIBILITÀ**"*
- ★ *"**ELIMINA L'IMPEGNO OPERATIVO** di gestire e monitorare l'**infrastruttura di base**"*

*"L'implementazione di microservizi con servizi completamente serverless **NON** comporta alcuna complessità operativa, e **NON** è necessario considerare alcun problema di scalabilità e disponibilità."*

**Metti questo a confronto con le Lezioni 4, 14 e 15:** scaling orizzontale, load balancer, Kubernetes, health probe, auto-scaling, self-healing... **nel serverless tutto questo è incluso nella piattaforma.** È il trade-off centrale: **rinunci al controllo in cambio dell'eliminazione di un'intera categoria di lavoro.**

**La composizione minima di un microservizio serverless**

★ *"Un'API creata con Amazon API Gateway e una funzione AWS Lambda sono **TUTTO CIÒ CHE SERVE** per fare un microservizio."*

*"Il team può usare questi servizi per **DISACCOPIARE** e **SEGMENTARE** l'ambiente **AL LIVELLO DI DETTAGLIO DESIDERATO.**"*

---

★ **Le 4 difficoltà dei microservizi che il serverless riduce**

*"Un ambiente di microservizi può introdurre le seguenti difficoltà:"*

| Difficoltà                                                                     | Come il serverless la riduce                                                                                                                                                        |
|--------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Overhead ripetuto per creare ogni nuovo microservizio                       | <i>"Il modello serverless <b>RIDUCE LA BARRIERA</b> di creazione di ogni microservizio successivo. Ad esempio, API Gateway consente persino di <b>CLONARE</b> le API esistenti"</i> |
| 2. Ottimizzazione della densità e dell'uso del server                          | ★ <i>"L'ottimizzazione dell'utilizzo dei server <b>NON È PIÙ RILEVANTE</b> con questo modello"</i>                                                                                  |
| 3. Complessità di eseguire più versioni di più microservizi contemporaneamente | Lambda supporta nativamente <b>versioning</b> e <b>alias</b> delle funzioni                                                                                                         |
| 4. Requisiti di codice lato client per integrarsi con molti servizi            | Gestito dall'API Gateway                                                                                                                                                            |

★ *"Quando si creano microservizi con risorse serverless, **questi problemi diventano MENO DIFFICILI** da risolvere e, in alcuni casi, **SEMPLICEMENTE SCOMPAIONO.**"*

---

## Mapping Microservice Architecture with AWS Serverless Services

Questa è la sezione più utile: **il confronto diretto** tra l'architettura cloud-native tradizionale (quella costruita nelle Lezioni 5-15) e la sua versione serverless.

**Cos'è "cloud native"**

**Definizione:** *"una metodologia industriale che consente alle aziende di **ripetere lo sviluppo e la distribuzione di applicazioni con opzioni di scalabilità più flessibili**, sfruttando appieno la potenza del cloud computing."*

"Le applicazioni cloud-native sono **orientate ai microservizi, CONTAINERIZZATE e ORCHESTRATE DINAMICAMENTE** per ottimizzare e utilizzare le risorse."

"Nella progettazione seguiamo **la metodologia 12-Factor App**."

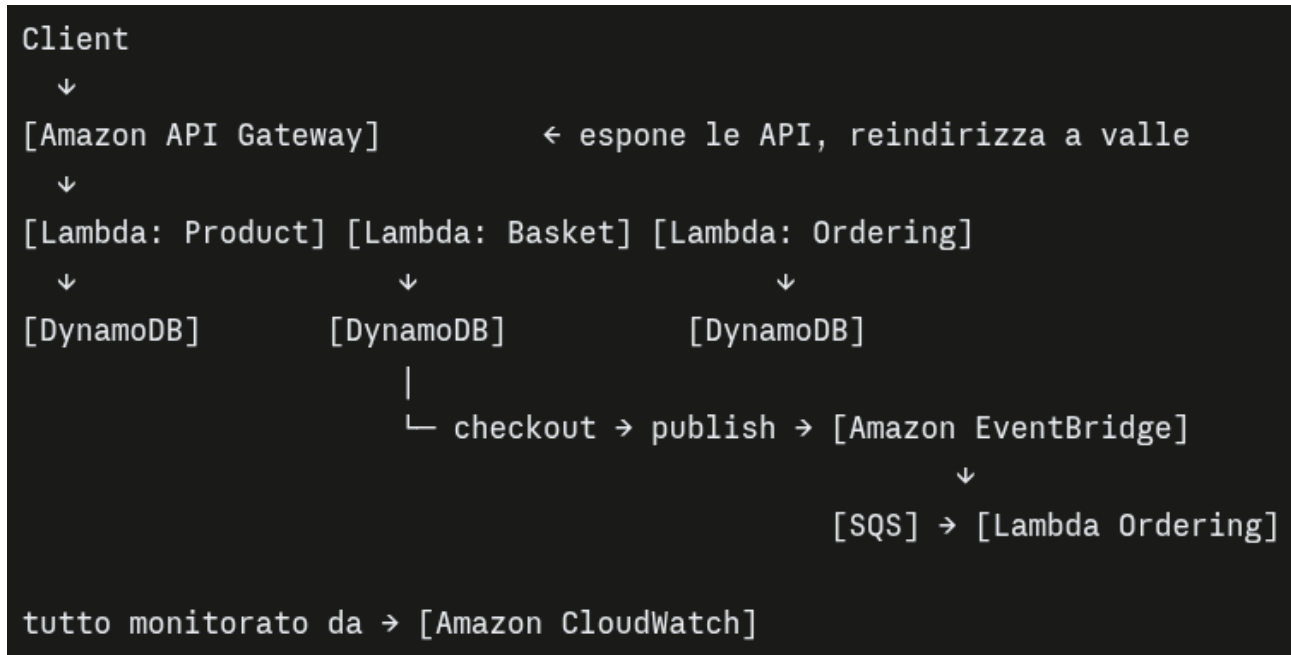
**Definizione: 12-Factor App** — una serie di 12 principi (codebase unica, dipendenze esplicite, configurazione nell'ambiente, servizi di backing come risorse allegate, processi stateless, port binding, concorrenza, disponibilità e arresto rapidi, parità dev/prod, log come stream di eventi, ecc.) per costruire applicazioni cloud-ready. Molti di questi principi li abbiamo incontrati implicitamente nel corso — soprattutto "**processi stateless**" (Lezione 4) e "**log come stream**" (Lezione 15).

---

## ★ Il confronto architetturale

| Componente     | Cloud-Native tradizionale (Lez. 5-15)                     | Serverless AWS (Lez. 16)                                                                                                                                                                         |
|----------------|-----------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| API Gateway    | <b>YARP</b> , Ocelot, nginx — <i>da gestire e scalare</i> | <b>Amazon API Gateway</b> — <i>"completamente serverless e gestito"</i>                                                                                                                          |
| Microservizi   | <b>Java Spring Boot</b> o <b>.NET in container Docker</b> | <b>AWS Lambda functions</b>                                                                                                                                                                      |
| Orchestrazione | ⚠ <b>Kubernetes</b>                                       | ✅ <b>nessuna</b> — gestita dalla piattaforma                                                                                                                                                     |
| Database       | PostgreSQL, SQL Server, MongoDB, Redis                    | <b>DynamoDB</b> (NoSQL key-value o documentale) — <i>"ma è possibile usare anche database relazionali serverless"</i>                                                                            |
| Message Broker | <b>RabbitMQ, Kafka</b> — <i>da installare e gestire</i>   | <b>Amazon EventBridge + SQS</b> — <i>"event bus serverless che fornisce tutte le funzionalità di message broker"</i>                                                                             |
| Monitoring     | Elastic Stack, Prometheus, Grafana                        | <b>Amazon CloudWatch</b> — <i>"registrazione e monitoraggio centralizzati"</i>                                                                                                                   |
| IaC            | <b>Terraform</b>                                          | <b>AWS CDK + CloudFormation</b>                                                                                                                                                                  |
| Scaling / HA   | ⚠ da progettare (Lez. 4, 14, 15)                          | ★ <i>"per impostazione predefinita, TUTTI questi servizi SCALANO AUTOMATICAMENTE e garantiscono alta disponibilità e affidabilità. Tutte queste caratteristiche vengono fornite DI DEFAULT."</i> |

## Il flusso serverless dell'e-shop



"Il checkout del carrello pubblica un evento su EventBridge, e questo verrà consumato da [Ordering] in modo asincrono."

## Event-Driven Serverless Microservices Design with AWS Lambda

### ★ Perché Lambda ed event-driven sono un'accoppiata naturale

"L'architettura event-driven offre agli sviluppatori **maggiore agilità, ELASTICITÀ, scalabilità e OTTIMIZZAZIONE DEI COSTI.**"

★ "AWS Lambda si adatta [perfettamente], poiché **la natura delle sue esecuzioni** [è basata su eventi]."

"**Lo scopo principale della funzione** [è di essere eseguita in risposta a un evento]. **Un evento ATTIVA una funzione.**"

### Le sorgenti di eventi

"Un evento che attiva una funzione può essere:"

- "una **richiesta HTTP** dall'API REST con **Amazon API Gateway**"
- "la gestione di un'**attività PIANIFICATA** da una **regola Amazon EventBridge**"
- "una **notifica**" [da un altro servizio]

★ "Ci sono **MOLTI eventi provenienti dalle risorse AWS**: anche **la più semplice applicazione** [può essere costruita su eventi]. **Ecco perché AWS Lambda** [è al centro dell'architettura]."

## Il principio event-driven

*"Le architetture event-driven si basano sulla **creazione di eventi in tutti i cambiamenti di stato dell'applicazione, che sono OSSERVABILI da altri servizi, ma il sistema di PUBBLICAZIONE degli eventi NON È A CONOSCENZA di quali consumatori [esistano].**"*

Identico al principio pub/sub della **Lezione 9** — il pattern non cambia, cambia solo l'implementazione.

## I design pattern già incorporati

★ *"I servizi AWS hanno GIÀ IMPLEMENTATO alcuni design pattern delle architetture distribuite, che possiamo utilizzare FACILMENTE SENZA INVESTIRE [tempo a costruirli]."*

Questa è forse l'osservazione più importante della lezione: **retry, dead letter queue, fanout, idempotenza, circuit breaker** — molte cose che nelle Lezioni 9, 12 e 15 abbiamo dovuto progettare a mano, **nel serverless sono funzionalità di configurazione dei servizi gestiti.**

## I 3 tipi di invocazione Lambda = i 3 tipi di comunicazione

*"I tipi di invocazione di AWS Lambda saranno il nostro percorso serverless... seguiremo questi tipi di invocazione event-driven COME TIPI DI COMUNICAZIONE DEI MICROSERVIZI."*

| Invocazione Lambda   | Tipo di comunicazione | Servizio    | Comportamento                                                                                                                  |
|----------------------|-----------------------|-------------|--------------------------------------------------------------------------------------------------------------------------------|
| Synchronous          | Sincrona              | API Gateway | "Il client* [invia una richiesta e attende la risposta]*"                                                                      |
| Asynchronous         | Asincrona             | EventBridge | "Il client invia una richiesta e <b>può ricevere una CONFERMA che l'evento è stato ricevuto, ma NON ottiene una risposta</b> " |
| Event Source Mapping | Polling               | SQS         | "I consumer* [ricevono] <b>un messaggio IN BATCH</b> "*                                                                        |

La chiusura

★ *"Fondamentalmente, **RAGGRUPPEREMO QUESTI EVENTI COME UN BOUNDED CONTEXT**, al fine di sviluppare\* [i microservizi]. Ciò significa che svilupperemo le funzioni Lambda **COME MICROSERVIZI.**"\**

Questa frase è il punto di arrivo concettuale del corso: **anche nel serverless, la decomposizione resta guidata dal dominio** — non dalla tecnologia. Le funzioni si raggruppano per **bounded context**, esattamente come i microservizi della Lezione 6.